5 & 3 // Result is 1 (0101 & 0011 = 0001)

-

| **(OR)**: The result is 1 if at least one bit is 1.

5 | 3 // Result is 7 (0101 | 0011 = 0111)

-

^ **(XOR)**: The result is 1 if the bits are different. Otherwise, the result is 0.

5 ^ 3 // Result is 6 (0101 ^ 0011 = 0110)

-

~ **(NOT)**: The result is the complement of the number (flips all the bits).

~5 // Result is -6 (in two's complement representation)

-

<< **(Left Shift)**: Shifts the bits of a number to the left by a specified number of positions. This is equivalent to multiplying the number by 2 raised to the power of the number of shifts.

5 << 1 // Result is 10 (0101 << 1 = 1010)

-